Data and Network Security

(Master Degree in Computer Science and Cybersecurity)

Lecture 4
Spring 2026

SAPIENZA
UNIVERSITÀ DI ROMA

Outline for today

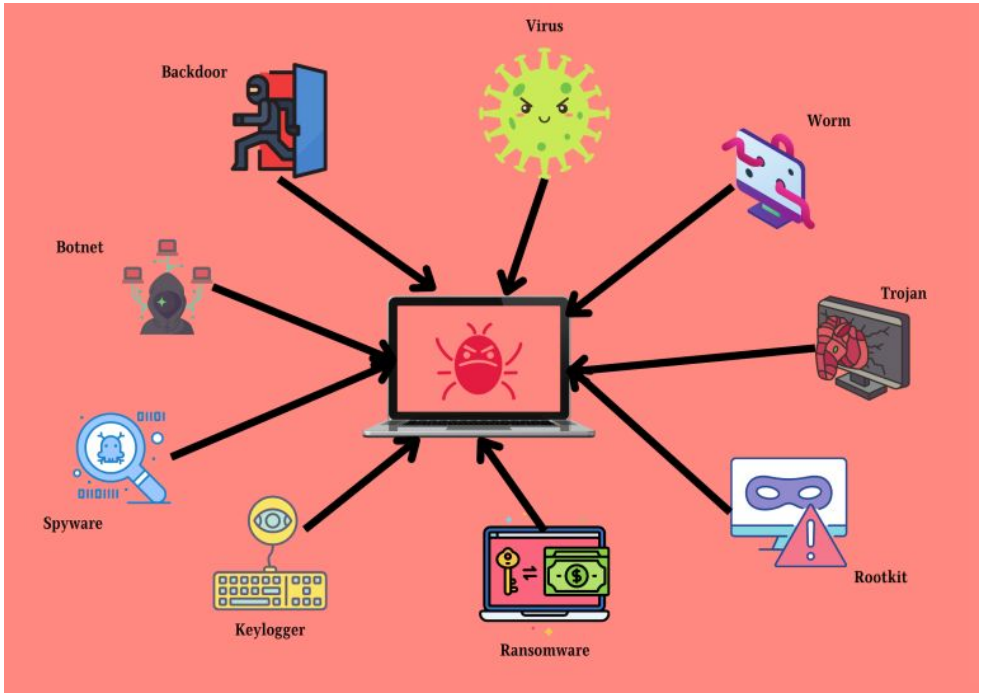
- Recap last lecture
- Covert channels
- Emerging threats

Outline for today

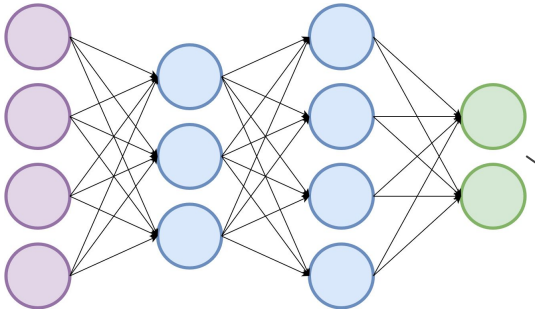
- **Recap last lecture**
- Covert channels
- Emerging threats

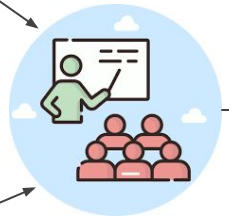
Malware

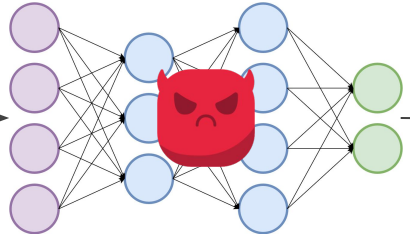
Malicious software designed to infiltrate, damage, or disrupt computer systems, networks, or devices, often with harmful intent.

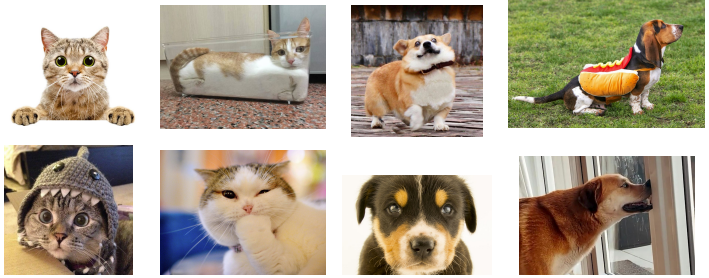


Hiding malware within Deep Neural Networks



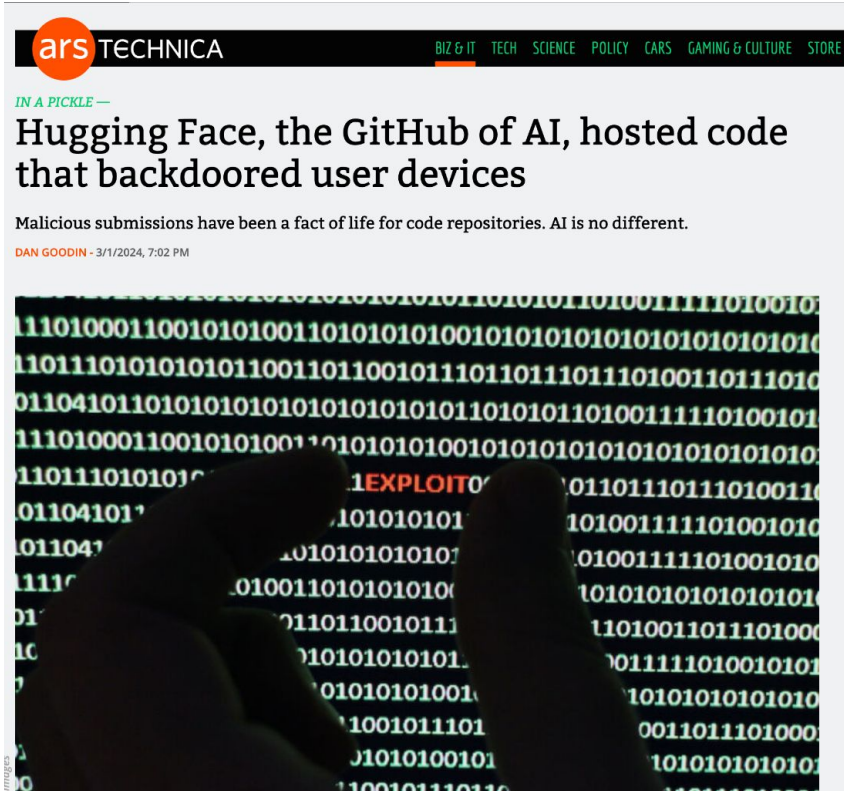






Fresh out of the oven (bad) news...

Hugging Face



ars TECHNICA

BIZ & IT TECH SCIENCE POLICY CARS GAMING & CULTURE STORE

IN A PICKLE —

Hugging Face, the GitHub of AI, hosted code that backdoored user devices

Malicious submissions have been a fact of life for code repositories. AI is no different.

DAN GOODIN - 3/1/2024, 7:02 PM

https://arstechnica.com/security/2024/03/hugging-face-the-github-of-ai-hosted-code-that-backdoored-user-devices/

Fresh out of the oven (bad) news...

Hugging Face

The Hacker News

Subscribe – Get Latest News

Home | Data Breaches | Cyber Attacks | Vulnerabilities | Webinars | Expert Insights | Contact

Malicious ML Models on Hugging Face Leverage Broken Pickle Format to Evade Detection

Feb 08, 2025 Ravie Lakshmanan Artificial Intelligence / Supply Chain Security

Cybersecurity researchers have uncovered two malicious machine learning (ML) models on Hugging Face that leveraged an unusual technique of "broken" pickle files to evade detection.

"The pickle files extracted from the mentioned PyTorch archives revealed the malicious Python content at the beginning of the file," ReversingLabs researcher Karlo Zanki said in a report shared with The

https://thehackernews.com/2025/02/malicious-ml-models-found-on-hugging.html?m=1

Fresh out of the oven (bad) news...

A Disney Worker Downloaded an AI Tool. It Led to a Hack That Ruined His Life.

Matthew Van Andel's experience reveals the threat that opportunistic hackers pose to companies and individuals

ı Matthew 'Dutch' Van Andel lost his job at Disney.

By Robert McMillan Follow *and Sarah Krouse* Follow | *Photographs by Adali Schell for WSJ*

Feb. 26, 2025 5:30 am ET

Share Resize 752 Listen (2 min)

The stranger messaging Matthew Van Andel online last July knew a lot about him—including details about his lunch with co-workers at Disney DIS 0.08% ▲ from a few days earlier.

His mind raced: he knew no one outside Disney would have access to that

THE WALL STREET JOURNAL.

Most Popular News

What Went Wrong at Saudi Arabia's Futuristic Desert Metropolis

The Retirement-Savings Weapon Doctors and Lawyers Use to Build Wealth

Markets Finally Woke Up to Tariff Reality. Is This a Buying Opportunity?

Your Phone Needs a Burner

https://www.wsj.com/tech/cybersecurity/disney-employee-ai-tool-hacker-cyberattack-3700c931

Practical implementation

```python
def inject(self, model, gamma: Optional[float] = None):
    start = time.time()

    model_st_dict = model.state_dict()
    models_w = []
    layer_lengths = dict()

    layers = [n for n in model_st_dict.keys() if "weight" in str(n)][:-1]
    for layer in layers:
        x = model_st_dict[layer].detach().cpu().numpy().flatten()
        layer_lengths[layer] = len(x)
        models_w.extend(list(x))

    models_w = np.array(models_w)
```

Practical implementation

```
np.random.seed(self.seed)
filter_indexes = np.random.randint(
    0, len(models_w), self.CHUNK_SIZE * self.chunk_factor * number_of_chunks, np.int32).tolist()

self.logger.info(
    f'Injecting on {self.CHUNK_SIZE * self.chunk_factor} * {number_of_chunks} = {self.CHUNK_SIZE * self.chunk_factor * number_of_chunks} parameters')
with tqdm(total=len(b)) as bar:
    bar.set_description('Injecting')
    current_chunk = 0
    current_bit = 0
    np.random.seed(self.seed)
    for bit in b:
        spreading_code = np.random.choice(
            a: [-1, 1], size=self.CHUNK_SIZE * self.chunk_factor)
        current_bit_cdma_signal = gamma * spreading_code * bit
        current_filter_index = filter_indexes[current_chunk * self.CHUNK_SIZE * self.chunk_factor:
                                              (current_chunk + 1) * self.CHUNK_SIZE * self.chunk_factor]
        models_w[current_filter_index] = np.add(
            models_w[current_filter_index], current_bit_cdma_signal)

        current_bit += 1
        if current_bit > self.CHUNK_SIZE * (current_chunk + 1):
            current_chunk += 1

        bar.update(1)

curr_index = 0
for layer in layers:
    x = np.array(
        models_w[curr_index:curr_index + layer_lengths[layer]])
    model_st_dict[layer] = torch.from_numpy(np.reshape(
        x, model_st_dict[layer].shape)).to(self.device)
    curr_index = curr_index + layer_lengths[layer]

end = time.time()
self.logger.info(f'Time to inject {end - start}')
return model_st_dict, len(b), len(self.payload), len(self.hash)
```

Practical implementation

```
np.random.seed(self.seed)
filter_indexes = np.random.randint(
    0, len(models_w_prev), self.CHUNK_SIZE * self.chunk_factor * number_of_chunks, np.int32).tolist()

x = []
ys = []

with tqdm(total=message_length) as bar:
    bar.set_description('Extracting')
    current_chunk = 0
    current_bit = 0
    np.random.seed(self.seed)
    for _ in range(message_length):
        spreading_code = np.random.choice(
            a: [-1, 1], size=self.CHUNK_SIZE * self.chunk_factor)
        current_filter_index = filter_indexes[current_chunk * self.CHUNK_SIZE * self.chunk_factor:
                                              (current_chunk + 1) * self.CHUNK_SIZE * self.chunk_factor]
        current_models_w_delta = models_w_delta[current_filter_index]
        y_i = np.matmul(spreading_code.T, current_models_w_delta)
        ys.append(y_i)

        current_bit += 1
        if current_bit > self.CHUNK_SIZE * (current_chunk + 1):
            current_chunk += 1

        bar.update(1)

y = np.array(ys)

np.random.seed(self.seed * 15)
preamble = np.sign(np.random.uniform(-1,  high: 1,  size: 200))

gain = np.mean(np.multiply(y[:200], preamble))
sigma = np.std(np.multiply(y[:200], preamble) / gain)
snr = -20 * np.log10(sigma)
self.logger.info(f'Signal to Noise Ratio = {snr}')
```

Outline for today

- Recap last lecture
- **Covert channels**
- Emerging threats

Covert Channel

Indirect communication channel between unauthorized parties that violates some security policy by using **shared resources** in a way in which these resources are not initially designed, bypassing mechanisms that do not permit direct communication between these unauthorized parties in the process.

As such, covert channels emerge as a threat to information-sensitive systems in which leakage to unauthorized parties may be unacceptable.

Covert channel types

- Storage
- Timing

Storage based covert channel

Covert channels that exploit storage resources to conceal data, often utilizing file attributes or reserved storage space.

Storage based covert channel

Covert channels that exploit storage resources to conceal data, often utilizing file attributes or reserved storage space.

- Data hidden within file (such as steganography)
- Modifying header fields

Storage based covert channel - Detection & Mitigation

- Monitoring file system activity
- Analyze file attributes
- Integrity checks to identify anomalies or unauthorized data storage
- Access control

Timing based covert channel

Covert channels that exploit variations in timing or delays within a standard communication channel to conceal data.

Modulating inter-packet delays

- Large delay - bit 1
- Small delay - bit 0

Timing based covert channel - Detection and Mitigation

Covert channels that exploit variations in timing or delays within a standard communication channel to conceal data.

- Detecting:
  - Modulated traffic would have different IDP distribution
- Mitigation
  - Injecting random delays
  - Buffering

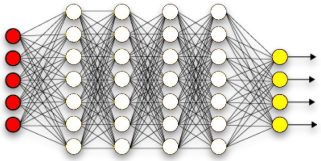
Outline for today

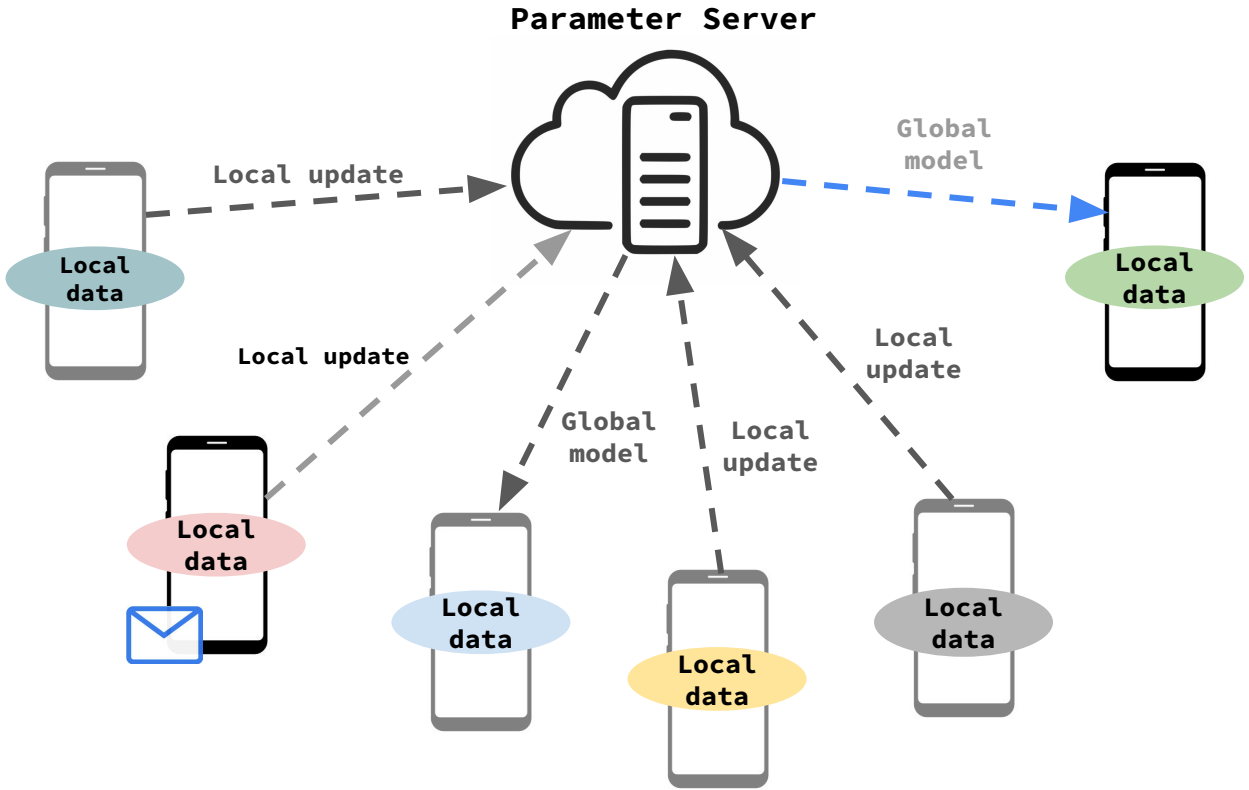
- Recap last lecture
- Covert channels
- **Emerging threats**

Covert Channels in Collaborative (federated) learning

Collaborative Learning

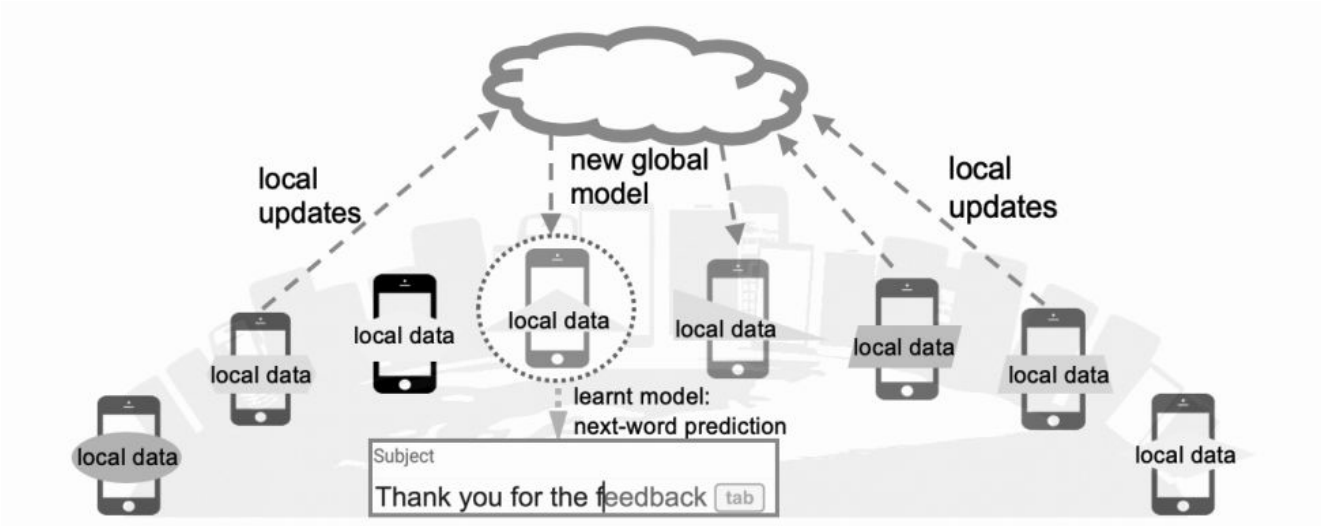
Collaborative Learning





What is Federated Learning?

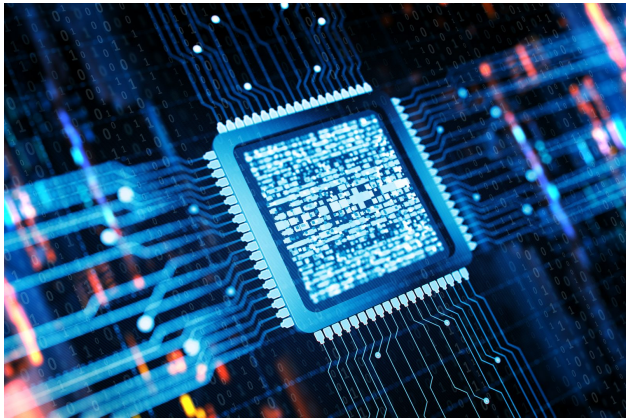
Google AI



Federated learning (FL) (also known as **collaborative learning**) is a machine learning technique that trains an algorithm across multiple decentralized edge devices or servers holding local data samples, without exchanging them.

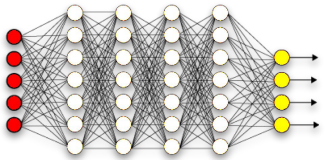
Why federated Learning?

- Data Privacy
- Low individual computing power
- Large collaborative computing power

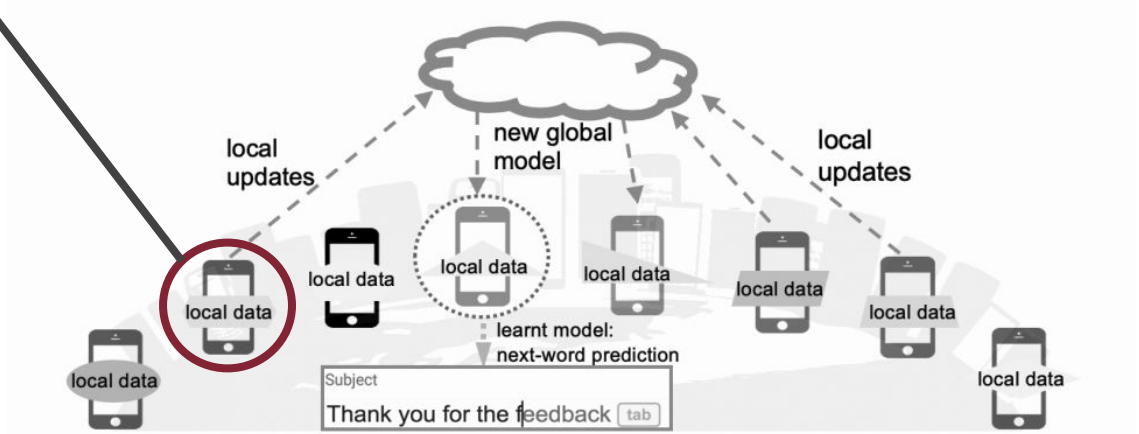


How does FL work? (local update)

$$\mathbf{W}_{t+1}^{k} = \mathbf{W}_t + \alpha \nabla \mathbf{W}_t^{k}$$

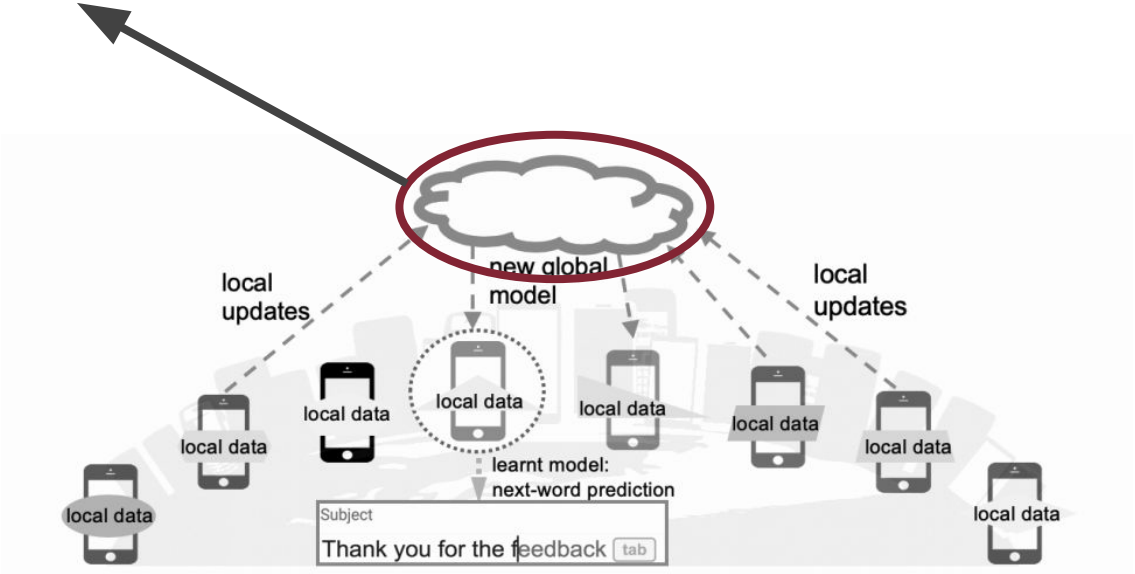


W



How does FL work? (global update)

$$\mathbf{W}_{t+1} = \mathbf{W}_t + \frac{\alpha}{n'} \sum_{k=1}^{n'} \nabla \mathbf{W}_t^k$$

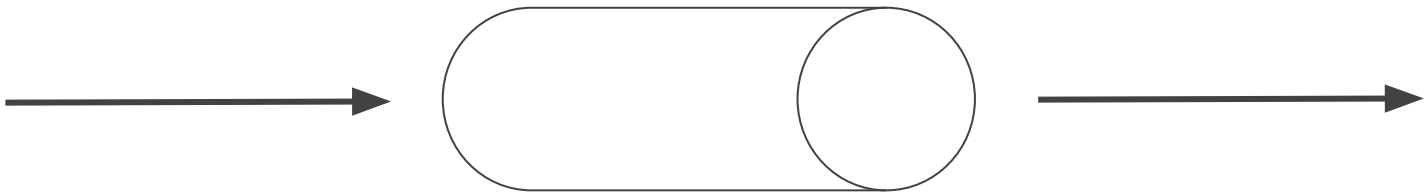


Digital (stealthy) communication

- Recap from lecture 2-

Digital Communication

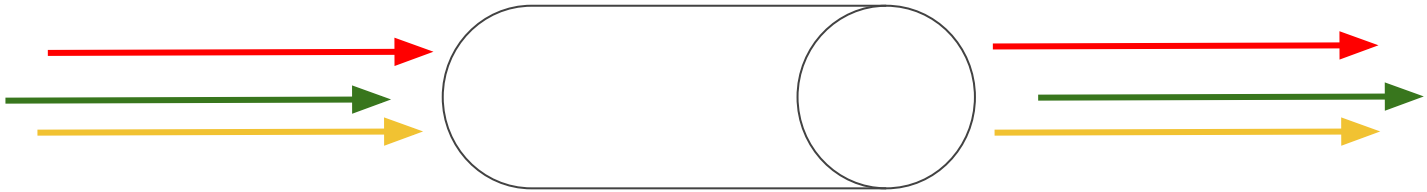
- Consists of the transfer of data or information using digital signals over a point-to-point channel.
- Data or information are digitally encoded as discrete signals, and are transferred through a communication channel.



Communication channel

What is CDMA?

- Channel coding technique by which a narrowband signal is spread into a wider bandwidth.
- Codes the data at a higher frequency by using pseudorandomly generated codes that the receiver knows.
- developed in the 1950s for military communications:
  - resist the enemy efforts to jam the communication channel.
  - hide the fact that the communication is taking place.



Communication channel

CDMA - example

Binary sequence: **[0, 1, 1]**

PSK **[-1, 1, 1]**

Spreading code: **[-1, 1, -1, -1, 1]**

Chip sequence: **[1, -1, 1, 1, -1, -1, 1, -1, -1, 1, -1, 1, -1, -1, 1]**

-5 **+5** **+5**

let's send some “message”

HowTo

Payload -> **P** bits **b** = $[b_0, \ldots, b_{P-1}]$

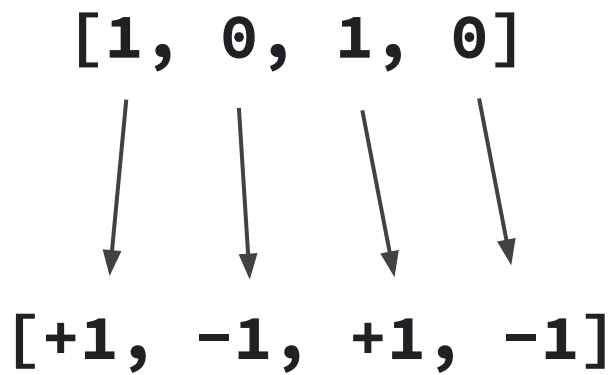
- Each bit is encoded as **±1**.

[1, 0, 1, 0]

HowTo (cont.)

Payload -> **P** bits **b** = $[b_0, \ldots, b_{P-1}]$

- Each bit is encoded as **±1**.



HowTo (cont.)

[1, 0, 1, 0]

[+1, -1, +1, -1]

Payload -> **P** bits **b** = [b_0, …,b_{P-1}]

- The spreading code of each bit (c_i) is a vector of ±1 that is of the same length of vector W, namely R.

[+1, −1, +1, −1]

[+1, +1, −1, +1, …,−1] c_i

R elements

HowTo (cont.)

Payload -> P bits b = [b_0, …,b_{P-1}]

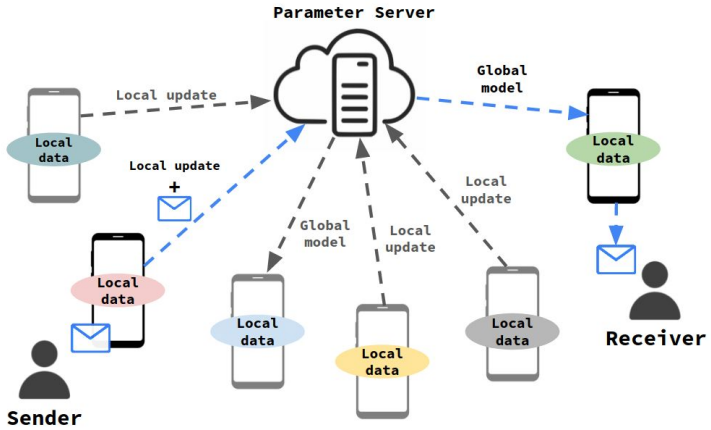
- **C** is an **R** by **P** matrix that collects all the codes.

	+1	+1	-1		+1
	-1	-1	-1		-1
	+1	-1	+1		+1
R	.	.	.		.
	.	.	.	. . .	.
	.	.	.		.
	-1	-1	-1		+1

P

“message” embedding

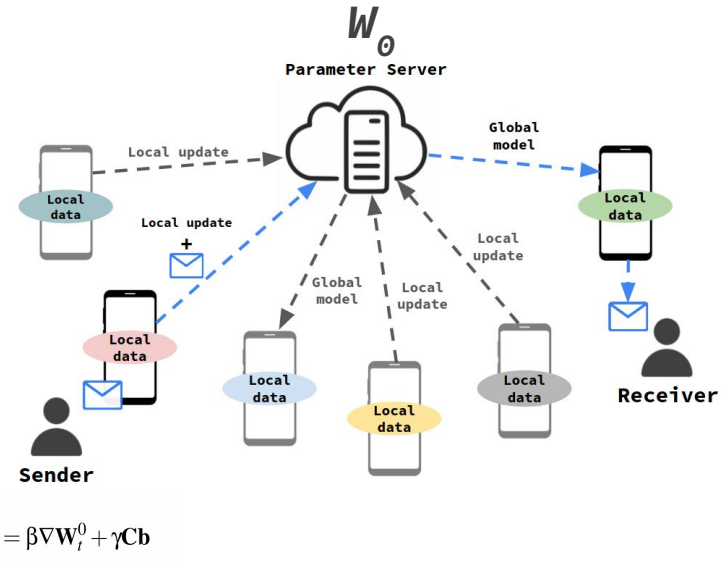
- **n** participants
- Parameter server proposes a set of weights W_0
- At each iteration, the participants use their local data to compute the gradient ∇W_t^k k = 0, …, n-1; t = 0, . . . T − 1



“message” embedding (cont.)

$$\widehat{\nabla \mathbf{W}_t^0} = \beta \nabla \mathbf{W}_t^0 + \gamma \mathbf{C}\mathbf{b}$$

The gradient update of the sender



γ and **β** are two gain factors to ensure that the message cannot be detected and that the power of the modified gradient is like the unmodified gradient for the other users.

“message” embedding (cont.)

How do we choose **γ** and **β** ?

$$\widehat{\nabla \mathbf{W}_t^0} = \beta \nabla \mathbf{W}_t^0 + \gamma \mathbf{C}\mathbf{b}$$

“message” embedding (cont.)

How do we choose **γ** and **β** ?

$$\widehat{\nabla \mathbf{W}_t^0} = \beta \nabla \mathbf{W}_t^0 + \gamma \mathbf{C}\mathbf{b}$$

β=0 and **γ=σ/√P**

- Our gradient would have the same power as the original.
- A hypothesis testing looking for a binomial or a Gaussian distribution will be able to detect that our gradient is not a true gradient.

“message” embedding (cont.)

How do we choose **γ** and **β** ?

$$\widehat{\nabla \mathbf{W}_t^0} = \beta \nabla \mathbf{W}_t^0 + \gamma \mathbf{C}\mathbf{b}$$

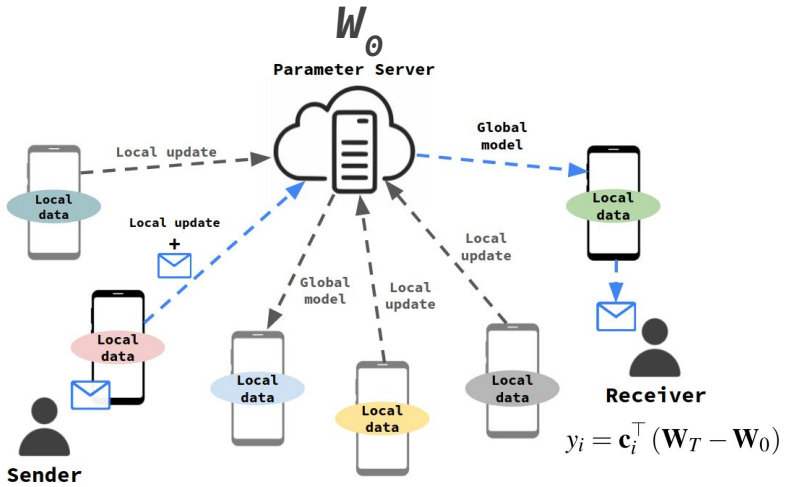
β=1 and **γ=0.1σ/√P**

- Our gradient will have the same distribution as the original gradient.
- The signal will be undetectable.

“message” extraction

To recover bit i of the payload:

$$y_i = \mathbf{c}_i^\top (\mathbf{W}_T - \mathbf{W}_0)$$



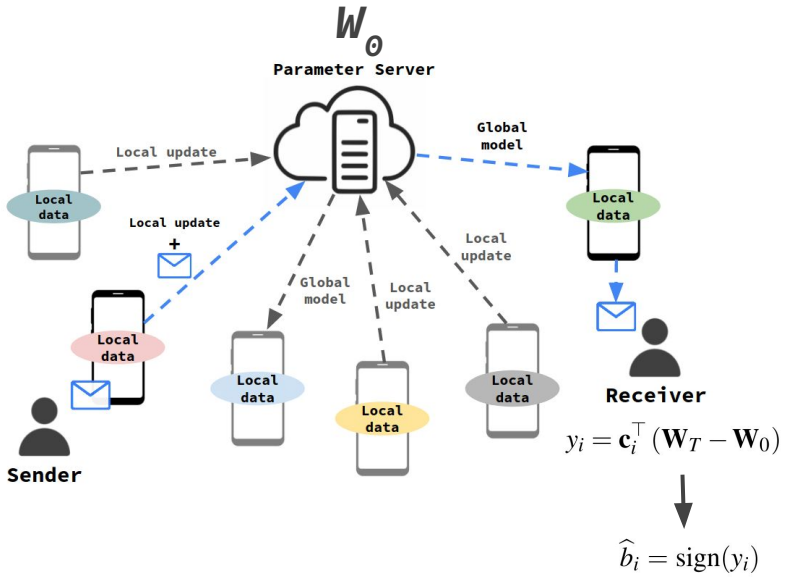
1. Bit i signal.
2. Noise from the gradients.
3. Noise from the other bits of the payload.

“message” extraction

To recover bit i of the payload:

$$y_i = \mathbf{c}_i^\top (\mathbf{W}_T - \mathbf{W}_0)$$

$$\widehat{b}_i = \text{sign}(y_i)$$



How we recover b_i ?

How we recover b_i ?

$$y_i = \mathbf{c}_i^\top (\mathbf{W}_T - \mathbf{W}_0)$$

Assume $\nabla \mathbf{W}_t^k$ is a zero-mean with a variance σ^2

How we recover b_i ?

$$
\begin{aligned}
y_i =& \mathbf{c}_i^\top \left(\mathbf{W}_T - \mathbf{W}_0\right) \\
=& \frac{\alpha}{n} \mathbf{c}_i^\top \left(\sum_{t=0}^{T-1} \left(\widehat{\nabla \mathbf{W}_t^0} + \sum_{k=1}^{n-1} \nabla \mathbf{W}_t^k \right) \right) \\
=& \frac{\alpha}{n} \left(T\gamma \mathbf{c}_i^\top \mathbf{C}\mathbf{b} + \mathbf{c}_i^\top \sum_{t=0}^{T-1} \left(\beta \nabla \mathbf{W}_0^k + \sum_{k=0}^{n-1} \nabla \mathbf{W}_t^k \right) \right) \\
=& \frac{\alpha}{n} \left(T\gamma \mathbf{c}_i^\top \mathbf{c}_i b_i + T\gamma \mathbf{c}_i^\top \mathbf{C}_{\neg i} \mathbf{b}_{\neg i} + \mathbf{c}_i^\top \widetilde{\mathbf{w}} \right) \\
=& \frac{\alpha}{n} \left(T\gamma \mathbf{c}_i^\top \mathbf{c}_i b_i + T\gamma \mathbf{c}_i^\top \widetilde{\mathbf{c}} + \mathbf{c}_i^\top \widetilde{\mathbf{w}} \right) \\
=& \frac{\alpha}{n} \left(T\gamma R b + \boldsymbol{\varepsilon}_i^{\mathbf{C}} + \boldsymbol{\varepsilon}_i^{\mathbf{W}} \right) = \frac{\alpha}{n} \left(T\gamma R b + \boldsymbol{\varepsilon}_i \right)
\end{aligned}
$$

How we recover b_i ?

$$
\begin{aligned}
y_i =& \mathbf{c}_i^\top \left(\mathbf{W}_T - \mathbf{W}_0\right) \\
=& \frac{\alpha}{n} \mathbf{c}_i^\top \left(\sum_{t=0}^{T-1} \left(\widehat{\nabla \mathbf{W}_t^0} + \sum_{k=1}^{n-1} \nabla \mathbf{W}_t^k \right) \right) \\
=& \frac{\alpha}{n} \left(T\gamma \mathbf{c}_i^\top \mathbf{C}\mathbf{b} + \mathbf{c}_i^\top \sum_{t=0}^{T-1} \left(\beta \nabla \mathbf{W}_0^k + \sum_{k=0}^{n-1} \nabla \mathbf{W}_t^k \right) \right) \\
=& \frac{\alpha}{n} \left(T\gamma \mathbf{c}_i^\top \mathbf{c}_i b_i + T\gamma \mathbf{c}_i^\top \mathbf{C}_{\neg i} \mathbf{b}_{\neg i} + \mathbf{c}_i^\top \widetilde{\mathbf{w}} \right) \\
=& \frac{\alpha}{n} \left(T\gamma \mathbf{c}_i^\top \mathbf{c}_i b_i + T\gamma \mathbf{c}_i^\top \widetilde{\mathbf{c}} + \mathbf{c}_i^\top \widetilde{\mathbf{w}} \right) \\
=& \frac{\alpha}{n} \left(T\gamma R b + \varepsilon_i^{\mathbf{C}} + \varepsilon_i^{\mathbf{W}} \right) = \frac{\alpha}{n} \left(T\gamma R b + \varepsilon_i \right)
\end{aligned}
$$

- ~c is a symmetric binomial distribution between **±(P - 1).**
- Binomial distribution with values between **±R(P-1).**
- For large **R** can be approximated by a zero-mean Gaussian with variance $\mathbf{T^2\gamma^2R(P-1)}$

How we recover b_i ?

$$
\begin{aligned}
y_i =& \mathbf{c}_i^\top (\mathbf{W}_T - \mathbf{W}_0) \\
=& \frac{\alpha}{n} \mathbf{c}_i^\top \left(\sum_{t=0}^{T-1} \left(\widehat{\nabla \mathbf{W}_t^0} + \sum_{k=1}^{n-1} \nabla \mathbf{W}_t^k \right) \right) \\
=& \frac{\alpha}{n} \left(T\gamma \mathbf{c}_i^\top \mathbf{C}\mathbf{b} + \mathbf{c}_i^\top \sum_{t=0}^{T-1} \left(\beta \nabla \mathbf{W}_0^k + \sum_{k=0}^{n-1} \nabla \mathbf{W}_t^k \right) \right) \\
=& \frac{\alpha}{n} \left(T\gamma \mathbf{c}_i^\top \mathbf{c}_i b_i + T\gamma \mathbf{c}_i^\top \mathbf{C}_{\neg i} \mathbf{b}_{\neg i} + \mathbf{c}_i^\top \widetilde{\mathbf{w}} \right) \\
=& \frac{\alpha}{n} \left(T\gamma \mathbf{c}_i^\top \mathbf{c}_i b_i + T\gamma \mathbf{c}_i^\top \widetilde{\mathbf{c}} + \mathbf{c}_i^\top \widetilde{\mathbf{w}} \right) \\
=& \frac{\alpha}{n} \left(T\gamma R b + \varepsilon_i^{\mathbf{C}} + \varepsilon_i^{\mathbf{W}} \right) = \frac{\alpha}{n} \left(T\gamma R b + \varepsilon_i \right)
\end{aligned}
$$

- Each component of $\widetilde{\mathbf{w}}$ adds up **$T(n-1+\beta)$** of these values
- By CLT (large enough **$T(n-1+\beta)$**) each one of these variables would be zero-mean Gaussian with a variance **$Tn\sigma^2$**.
- When we multiply this vector by **$\mathbf{c}_i$** and add all the components together, we end up with a zero-mean Gaussian with a variance **$RTn\sigma^2$**, because the components of **$\mathbf{c}_i$** are ±1.

How we recover b_i?

How we recover b_i?

$$
\begin{aligned}
y_i = \mathbf{c}_i^\top (\mathbf{W}_T - \mathbf{W}_0) &= \frac{\alpha}{n}\left(T\gamma \mathbf{c}_i^\top \mathbf{c}_i b_i + T\gamma \mathbf{c}_i^\top \mathbf{C}_{\neg i}\mathbf{b}_{\neg i} + \mathbf{c}_i^\top \widetilde{\mathbf{w}}\right) \\
&= \frac{\alpha}{n}\left(T\gamma \mathbf{c}_i^\top \mathbf{c}_i b_i + T\gamma \mathbf{c}_i^\top \widetilde{\mathbf{c}} + \mathbf{c}_i^\top \widetilde{\mathbf{w}}\right) \\
&= \frac{\alpha}{n}\left(T\gamma R b + \varepsilon_i^{\mathbf{C}} + \varepsilon_i^{\mathbf{W}}\right) = \frac{\alpha}{n}\left(T\gamma R b + \varepsilon_i\right)
\end{aligned}
$$

- **C**, **b** and $\nabla \mathbf{W}_t^k$ are mutually independent, thus $\boldsymbol{\varepsilon}_i$ is zero mean with a variance that it is the sum of the variances of $\boldsymbol{\varepsilon}^{\mathbf{C}}_i$ and $\boldsymbol{\varepsilon}^{\mathbf{W}}_i$ and also Gaussian distributed.
- The distribution of $\mathbf{y}_i$ is given by $\mathbf{y}_i \sim \mathcal{N}(\mathbf{T\gamma R b}_i, \mathbf{T^2\gamma^2 R(P-1) + RTn\sigma^2})$, and if normalize by **TγR**:

$$
y_i \sim \mathcal{N}\left(b_i, \frac{T^2\gamma^2 R(P-1) + TRn\sigma^2}{T^2\gamma^2 R^2}\right)
$$

$$
\mathcal{N}\left(b_i, \frac{P-1}{R} + \frac{n\sigma^2}{TR\gamma^2}\right)
$$

How we recover b_i?

$$y_i \sim \mathcal{N}\left(b_i, \frac{T^2\gamma^2 R(P-1) + TRn\sigma^2}{T^2\gamma^2 R^2}\right)$$

$$\mathcal{N}\left(b_i, \frac{P-1}{R} + \frac{n\sigma^2}{TR\gamma^2}\right)$$

Using a long enough error-correcting code, we can ensure errorless communication when the variance is about **1**.

β=1 and **γ=0.1σ/√P**

$$\frac{P-1}{R} + \frac{n\sigma^2}{TR\gamma^2} = \frac{P-1}{R} + \frac{n\sigma^2}{TR\left(\frac{0.1\sigma}{\sqrt{P}}\right)^2}$$

$$= \frac{P-1}{R} + \frac{100nP\sigma^2}{TR} \approx \frac{(T+100n)P}{TR}$$

- At least **T>100nP/(R-P)** rounds before the message can be decoded.

How we recover b_i faster?

- We incorporate multiple senders

How we recover b_i faster?

- We incorporate multiple senders

  - The y_i distribution will become:

$$\mathcal{N}(MT\gamma Rb_i, M^2T^2\gamma^2R(P-1) + RTn\sigma^2)$$

How we recover b_i faster?

- We incorporate multiple senders

  - The y_i distribution will become:

$$\mathcal{N}(MT\gamma Rb_i, M^2T^2\gamma^2R(P-1)+RTn\sigma^2)$$

$T>nP/M^2(R-P)$ M^2 times faster

The pillars of evaluation

Stealthiness of the communication

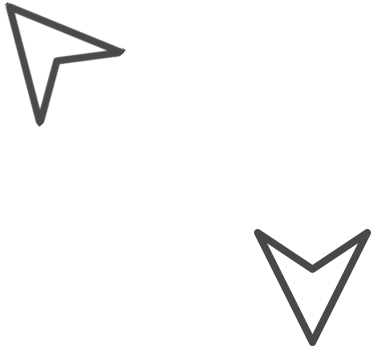
Impact on model performance

“Message” delivery time

The pillars of evaluation

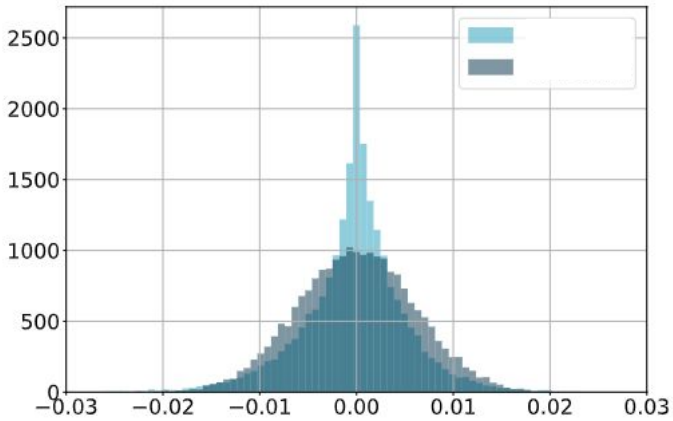
Stealthiness of the communication

Impact on model performance



“Message” delivery time

Stealthiness of communication



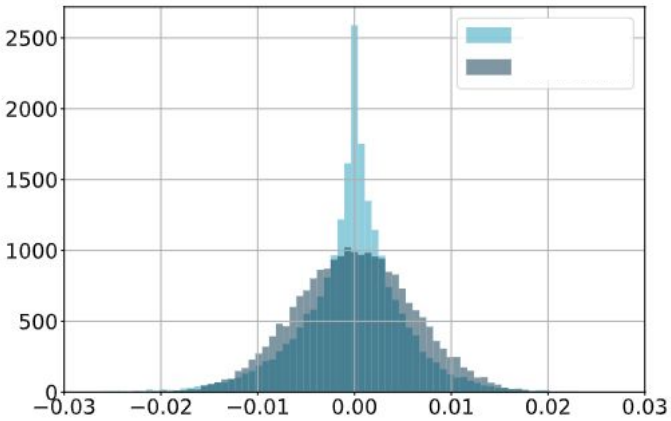
Regular gradient update

vs.

Non-Stealthy gradient update

In **non-stealthy** we are not sending a gradient that is useful for learning, instead we send our signal with the same power as our gradient would have.

Stealthiness of communication



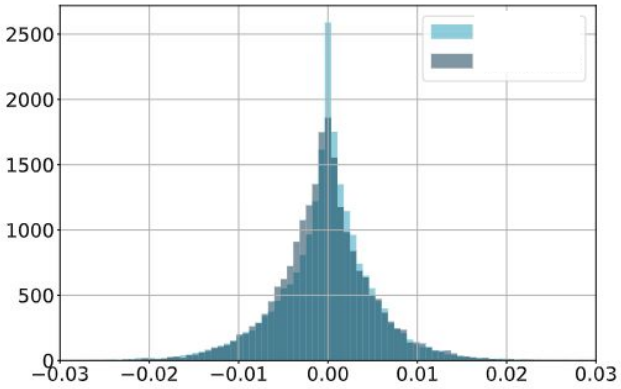
Regular gradient update

vs.

Non-Stealthy gradient update

NON-STEALTHY **might** be detectable in cases where the global parameter server can observe individual gradient updates.

Stealthiness of communication



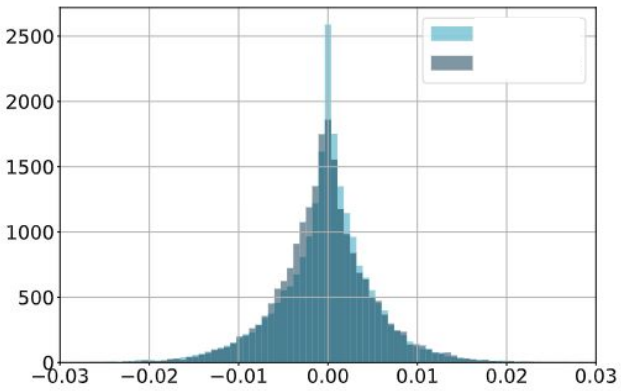
Regular gradient update

vs.

Full-Stealthy gradient update

In **full-stealthy** we are sending a gradient that is useful for learning, and buried into it we put also our signal.

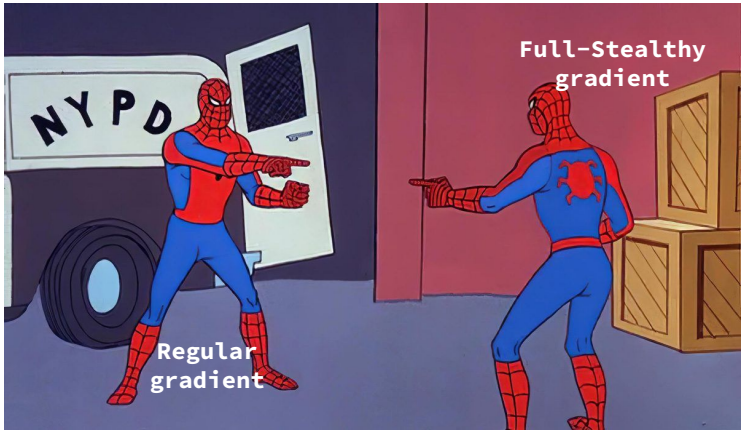
Stealthiness of communication



Regular gradient update

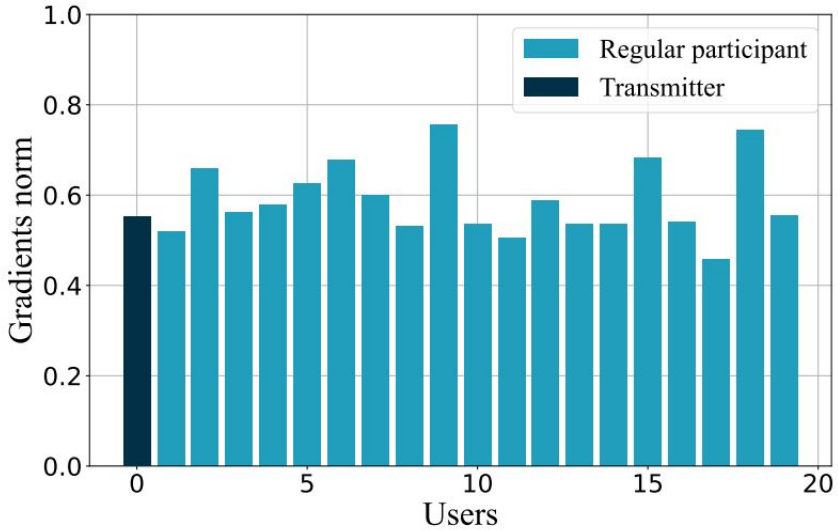
vs.

Full-Stealthy gradient update



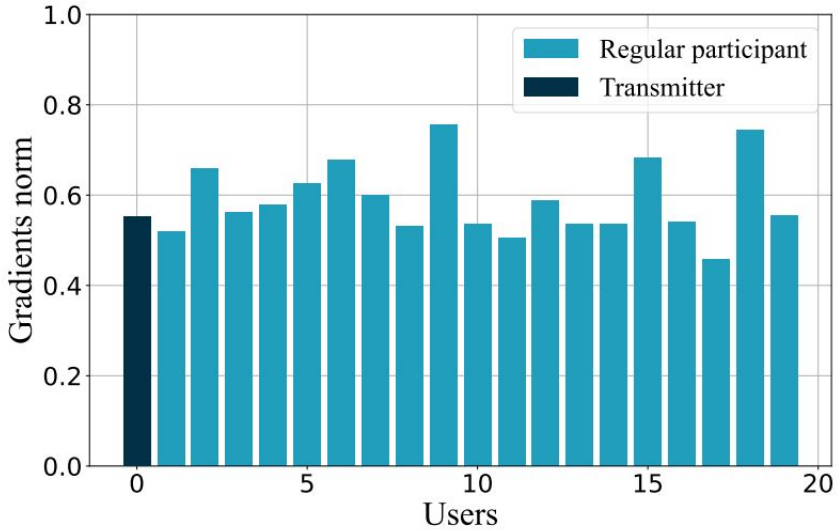
Gradients are statistically indistinguishable

Stealthiness of communication (cont.)



Parameter server observes individual gradient updates

Stealthiness of communication (cont.)

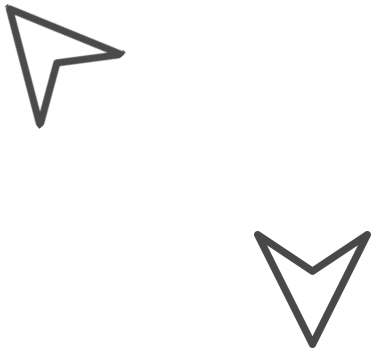


Parameter server observes individual gradient updates

The pillars of evaluation

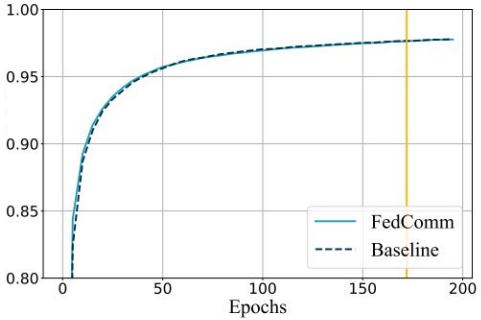
Stealthiness of the communication

Impact on model performance



“Message” delivery time

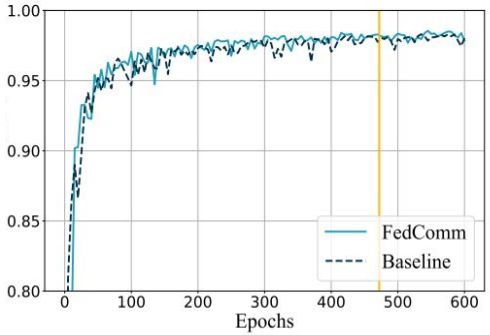
Impact on the model performance



100 participants

10 senders FULL Stealthy

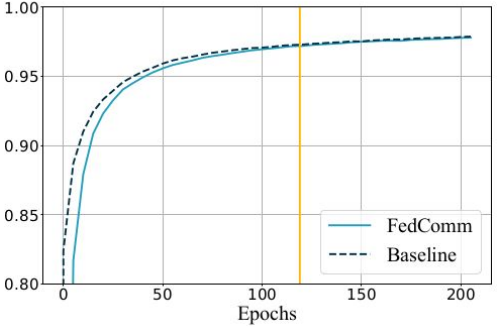
100% aggregated per round



100 participants

10 senders FULL Stealthy

20% aggregated per round



100 participants

1 sender NON Stealthy

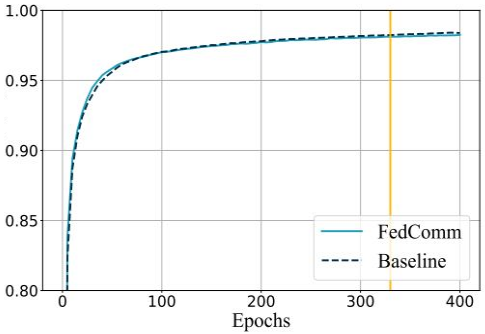
100% aggregated per round

The pillars of evaluation

Stealthiness of the communication

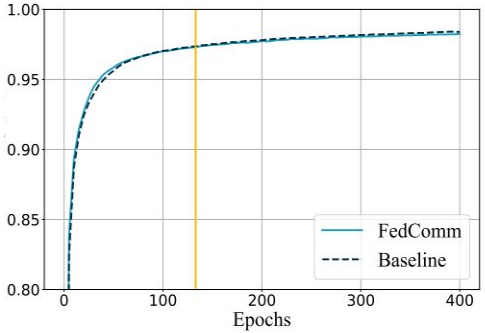
“Message” delivery time

“message” delivery time



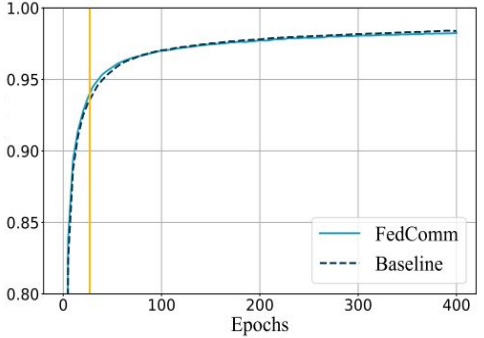
1 sender

100% aggregated per round



2 senders

100% aggregated per round

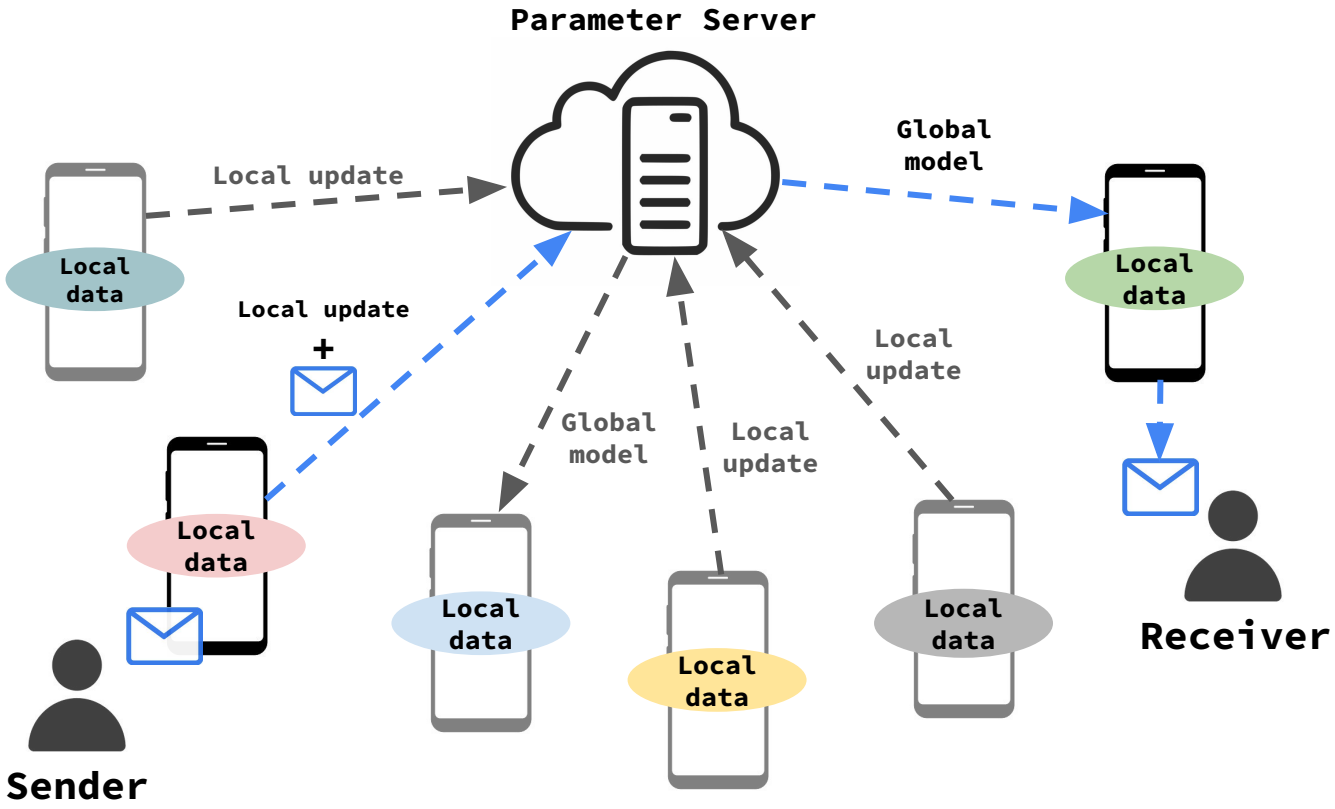


4 senders

100% aggregated per round

- Everything else held equal.

Remarks


Parameter Server
Local update
Global model
Local data
Local update
+
Local data
Global model
Local update
Local update
Local data
Local data
Local data
Local data
Sender
Receiver


That's all Folks!

Reading Material

1. Covert Channels (Concepts and definitions): Link
2. Covert communication in collaborative learning: Link-1, Link-2 (research papers).